



10Power IoT Fellowship

in collaboration with **NESL** (NorthEast Smart Labs)

Continuous Residual-Current (Leakage) Detection for Distributed Energy Management

Integrating the IVY MD0630 Type-B RCM into the OpenAMI / MeshEMS platform:
from Modbus bring-up to multi-EMS fault isolation and
reverse-flow-adaptive protection

Consolidated technical report — Sprints S1 to S6

Author

Allahaddje Bonheur
IoT Fellow, 10Power
ESP32 firmware & embedded integration

Technical supervision

Glenn — NESL

Target platform

NESL EMS controller (ESP32-S3, PCB 865B)
Firmware branch `modbus-testing`

July 2026

Executive summary

Residual (leakage) current is the direct physical signature of insulation failure. When it flows to earth it is, at once, a **fire and electrocution hazard** and an early **diagnostic of degrading equipment**. Modern low-voltage sites — with photovoltaics, battery storage and EV charging — inject a **DC component** into that leakage, which blinds the ordinary AC-only residual-current device (RCD). Detecting it requires a **Type-B residual-current monitor (RCM)** and, to make the measurement *actionable*, an intelligent controller that watches it continuously.

This report documents the design, implementation and hardware validation of exactly that capability for the **OpenAMI / MeshEMS** energy-management platform (NESL). Over six sprints (S1–S6) the **IVY MD0630 Type-B AC+DC RCM** was integrated end-to-end into the ESP32-S3 EMS firmware: its Modbus protocol was reverse-engineered and confirmed (S1); the live AC/DC leakage is read on the EMS (S2); the life-safety thresholds are written and verified at commissioning (S3); leakage changes are correlated with energy events (S4); a literature-grounded, multi-EMS **nomination-and-isolation** scheme locates a leak on a shared feeder and streams it over MQTT (S5); and the module’s **hardware alarm outputs** are sensed while the AC threshold is **adaptively tightened under reverse power flow** (S6).

The work follows the supervisor’s “**path of least blockage**” engineering method — build with configurable parameters and a mock pipeline so progress never stalls on an unknown, then confirm each assumption against real hardware. Every sprint is proven: the register map was confirmed against the vendor tool; live reads ran 12/12 with zero failures; threshold writes were traced to the correct Modbus function code; and the S6 alarm path was flashed and validated on the bench — a process that uncovered and fixed three defects only real hardware reveals (pin availability, alarm polarity, and sampling latency). The result is a complete, tested, and documented leakage-detection subsystem, ready for feeder-scale deployment.

Table of contents

Executive summary	2
Nomenclature	4
1 Introduction, context and objectives	4
1.1 Why leakage current matters	4
1.2 The host platform: OpenAMI / MeshEMS	4
1.3 Engineering method: the “path of least blockage”	5
1.4 Objectives and sprint structure	5
2 S1 — Modbus bring-up and register-map derivation	6
2.1 Problem	6
2.2 Method and result	6
3 S2 — Live AC/DC read on the EMS	7
3.1 Objective	7
3.2 Method and result	7
4 S3 — Config-time threshold writes	8
4.1 Objective	8
4.2 The investigation	8
5 S4 — Energy-change correlation	9
5.1 Rationale	9
5.2 Design	10
6 S5 — Multi-EMS nomination and MQTT isolation	10
6.1 Objective and literature grounding	10
6.2 Implementation and evidence	11
7 S6 — Hardware alarm sense and reverse-flow-adaptive protection	12
7.1 (B) Reverse-power-flow adaptive threshold	13
7.2 (A) Hardware alarm sense — and its hardware validation	13
8 Consolidated results and validation	14
9 Discussion, limitations and perspectives	14
References	15

Nomenclature

Term	Meaning
AMI	Advanced Metering Infrastructure
CRC	Cyclic Redundancy Check (Modbus frame integrity)
DC/AC	Direct / Alternating current component of the residual current
EMS	Energy Management System (the NESL ESP32-S3 controller)
FC	Modbus Function Code (FC03 read, FC06 write-single, FC16 write-multiple)
FLISR	Fault Location, Isolation and Service Restoration
GPIO	General-Purpose Input/Output pin
LV	Low Voltage (distribution level)
MQTT	Message Queuing Telemetry Transport (publish/subscribe messaging)
Modbus RTU	Serial fieldbus protocol used by the MD0630
PV / EV	Photovoltaic / Electric Vehicle
RCD / RCM	Residual-Current Device (protective) / Monitor (measuring)
RCM Type B	Monitor sensing AC <i>and</i> DC residual current
RS-485 / UART	Serial physical / logic layers
I_{res} / V_{res}	Residual current / residual voltage

1 Introduction, context and objectives

1.1 Why leakage current matters

In a healthy installation, the current leaving through the live conductor returns entirely through the neutral; their vector sum is zero. **Residual (leakage) current** is the small imbalance that appears when part of the current finds an unintended path to earth — through degraded insulation, moisture, or a person in contact with a fault. It is therefore both a **safety** quantity (fire, electric shock) and a **condition-monitoring** quantity (insulation ageing, water ingress, developing faults).

Classical protection uses a **Type-AC or Type-A RCD**, which only sees alternating residual current. The modern LV site breaks that assumption: PV inverters, battery converters and EV chargers can inject a **smooth DC residual current**, which can magnetically **saturate a conventional RCD's core and prevent it from tripping** — a documented safety gap addressed by the Type-B classification [1, 2]. A **Type-B RCM** measures both the AC and the DC residual current and is the correct sensor for a site with power electronics on it.

Measuring the leakage, however, is only half of the problem. A bare RCM trips its own relay locally; it does not tell a distributed energy system *where* on a shared feeder the leak is, whether it is worsening, or how it correlates with what the site is doing. Turning the measurement into **actionable intelligence** — detection, localisation, isolation and adaptive protection — is the contribution of this work.

1.2 The host platform: OpenAMI / MeshEMS

The sensor is not a standalone product; it is a subsystem of the NESL **Energy Management System (EMS)** — an ESP32-S3 controller (PCB 865B) that already meters energy, controls relays and speaks MQTT within the OpenAMI / MeshEMS ecosystem (Street-, Build- and

DTM-EMS variants). Multiple EMS share a low-voltage feeder, which is precisely what makes **coordinated**, multi-node leakage isolation both possible and valuable.

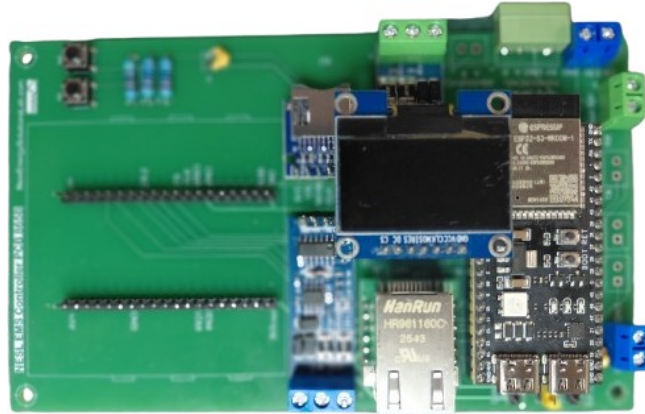


Figure 1: The NESL Street-EMS controller — the class of ESP32-S3 device into which the leakage subsystem is integrated (one of several EMS variants sharing an LV feeder).

1.3 Engineering method: the “path of least blockage”

The assignment was run under two directives from the supervisor that shaped every design decision:

1. **Analyse, design, review — then code.** Each sprint began with a written design note and a review, not with firmware. This report is the consolidation of those notes.
2. **Never stay blocked.** Where a fact was unknown (e.g. the exact register offsets), the driver was built with **configurable parameters** and a **mock data pipeline**, so the whole chain (data model → MQTT → insights) could run and be tested *before* the hardware answer arrived — and the assumption was then confirmed against the real device.

This method is visible throughout: mock ramps that stand in for a rising leak, build flags that isolate each capability, and a discipline of **confirming every assumption against real hardware or the vendor’s own tool** before treating it as fact.

1.4 Objectives and sprint structure

General objective. Integrate a Type-B AC+DC leakage monitor into the EMS firmware so the platform can *continuously* detect, localise and act on residual-current faults across a shared feeder.

The work was decomposed into six sprints, summarised in Table 2 and detailed in the following sections.

Table 2: The six sprints of the leakage-detection assignment.

Sprint	Objective	Outcome
S1	Modbus bring-up + derive the register map	Map derived and confirmed vs vendor tool
S2	Read live AC/DC leakage on the EMS	12/12 reads, 0 failures

Sprint	Objective	Outcome
S3	Write life-safety thresholds, with read-back	Solved via FC16 (not FC06); verified
S4	Correlate leakage changes with energy events	Cache implemented; exercised end-to-end
S5	Multi-EMS nomination + MQTT isolation	Literature-grounded; proven on device + MQTT
S6	Hardware alarm sense + reverse-flow threshold	Implemented; alarm path verified on HW

2 S1 — Modbus bring-up and register-map derivation

2.1 Problem

The MD0630 is documented as a Modbus RTU device, but its datasheet gives **no register offsets, no function code and no scaling** — the exact information a driver needs. S1’s goal was to establish serial communication and derive that map, first on a PC bench (per the “path of least blockage”: prove the protocol before committing the EMS bus).

2.2 Method and result

The module was powered at **12 V** and connected to the PC through a USB-TTL adapter acting as the Modbus master. The single detail that unblocked communication was electrical: the module’s UART is **open-drain and needs pull-ups**, and — because the lab adapter exposed no VCC wire — those pull-ups were fed from an ESP32 5 V pin, **with every ground tied to a single common ground** (Figure 2).

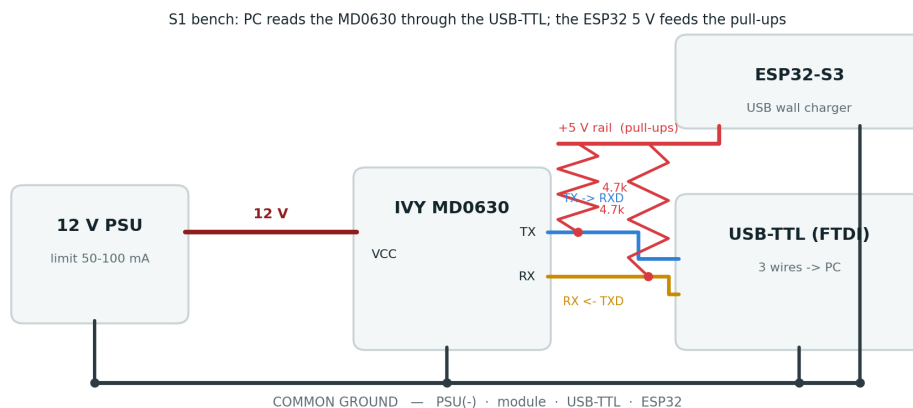


Figure 2: S1 bench wiring — 12 V powers the module; the PC reads it over the USB-TTL; the ESP32 5 V feeds the pull-ups; every ground is common.

Once the link was live (address 1, 9600 8N1), the register block was found by scanning — a 7-wide read returned *illegal data address* (0x02), narrowing the valid block to 0x0000–0x0005 —

and every field was then **cross-checked against IVY’s own CT.exe tool**, whose raw frames are authoritative.

Table 3: Derived MD0630 register map (FC 0x03, scaling raw $\times 0.1$ mA). Defaults match the datasheet: DC 6 mA, AC 30 mA.

Register	Default (raw / $\times 0.1$ mA)	Field	Access
0x0000	0 / 0.00	DC leakage current	R
0x0001	0 / 0.00	AC leakage current	R
0x0002	60 / 6.0	DC threshold	R/W
0x0003	300 / 30.0	AC threshold	R/W
0x0100	1	Version	R
0x0111	5	Minor version	R

S1 — key result

Communication established at **address 1, 9600 8N1, FC 0x03, scaling raw $\times 0.1$ mA**. The register map (3) was derived from the live device and **independently confirmed against the vendor’s CT.exe frames** — resolving it as neither the earlier proposed specification nor the raw configuration-file field order.

3 S2 — Live AC/DC read on the EMS

3.1 Objective

Move the module off the PC and have the **EMS itself** read the live residual current over its own serial bus, feeding the values into the firmware’s energy data model (`leakageModel`, published on the `subpanel_RCMleaks` topic).

3.2 Method and result

A useful finding simplified the wiring: the MD0630 exposes **plain UART logic**, not RS-485 differential, so it connects **directly** to the ESP32 UART pins with no transceiver (Table 4). Pull-ups go to **3.3 V** — the ESP32-S3 GPIOs are **not** 5 V-tolerant, and the open-drain TX means the pull-up rail sets the (safe) logic-high level.

Table 4: S2 bench wiring — direct UART, no transceiver.

Module	ESP32-S3	Note
TX	GPIO42 (RS485_1 RX)	+ 4.7 k Ω pull-up to 3V3
RX	GPIO7 (RS485_1 TX)	+ 4.7 k Ω pull-up to 3V3
GND	GND	common ground (EMS + module + 12 V PSU—)
VCC	12 V PSU	never the ESP32

S2 bench: ESP32 reads the MD0630 directly on its UART (GPIO42/7); 3.3 V feeds the pull-ups

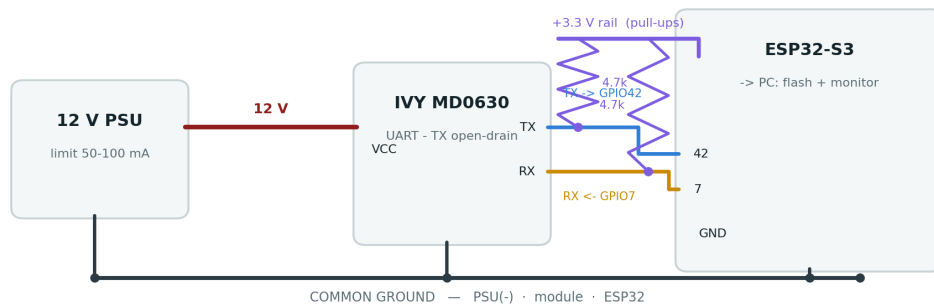


Figure 3: S2 bench wiring — the EMS reads the MD0630 directly on its UART (GPIO42/7); the module runs on 12 V; the ESP32 3.3 V feeds the pull-ups; every ground is common.

Two defects were found and fixed, both instructive:

- **Wrong pins.** The wires were first landed on the board’s silkscreen “RX/TX”, which are the **USB console UART** (GPIO44/43), not the Modbus bus (GPIO42/7) — the read simply timed out until the wires were moved to the numbered pins.
- **Address collision.** The on-board SHT20 temperature sensor is also polled at **address 1**, so the two pollers collided on the shared bus and reads were intermittent; a bring-up build flag that skips the SHT20 poll produced reliable reads. (In production the CT is commissioned to node 100, so the two coexist.)

S2 — key result

The EMS reads the physical module reliably: **12 consecutive successful reads, 0 failures**, both channels 0.00 mA at rest (no induced leakage). S1’s derived map is thereby validated on the target hardware, not merely on the PC bench.

4 S3 — Config-time threshold writes

4.1 Objective

Write the module’s **life-safety thresholds** — DC 6 mA (0x0002), AC 30 mA (0x0003) — at commissioning, **always confirmed by a read-back**, and never from the polling loop.

4.2 The investigation

The first write attempt exposed a classic embedded pitfall. An **FC 0x06 (write single register)** was *accepted* by the module — no Modbus exception was returned — yet the register **never changed**. Only the read-back safety caught it (wrote 27 mA, read back 30 mA, rejected the write). Rather than assume an undocumented “unlock” sequence, the vendor tool was **sniffed** performing a write over the USB-TTL: its frames revealed the answer.


```
Write : 00 10 00 03 00 01 02 01 0E 2B A7  <- FC 0x10, reg 0x0003, qty 1, value 0x010E = 27
Read  : 00 10 00 03 00 01 F0 18          <- success; NO unlock frame anywhere
```

The MD0630 **silently ignores FC 0x06** and **requires FC 0x10 (write multiple registers)** — no unlock exists. The driver’s `writeThreshold_mA()` now writes via FC16 and enforces a read-back:

Table 5: The write is never trusted without a matching read-back.

Step	Action
1	<code>setTransmitBuffer(0, mA × 10)</code> — e.g. 30 mA → 300 = 0x012C
2	<code>writeMultipleRegisters(reg, 1)</code> — FC 0x10 to 0x0002 (DC) or 0x0003 (AC)
3	40 ms settle, then read back (FC 0x03)
4	Confirm: read-back == written → OK; else reject (0xE4)

On the EMS this swept AC 30 → 27 → 30 mA with matching read-backs, then restored the safe factory defaults (DC 6 / AC 30 mA).

S3 — key result

Threshold writes work via **FC 0x10 with mandatory read-back**, verified on the EMS. The earlier “unlock required” hypothesis was **wrong** — the blocker was the function code (FC06 vs FC16), established by tracing the vendor tool rather than guessing.

Per the supervisor’s safety rules, writes are **config-time only**, target **only** the two confirmed threshold registers, always read back and reject on mismatch, and the vendor tool’s *Calibration* function (factory-only) is never invoked.

5 S4 — Energy-change correlation

5.1 Rationale

A leak rarely appears in isolation from what the site is *doing*: a step in residual current that coincides with a load switching on, a PV ramp, or an EV session is far more diagnostic than a bare number. S4 builds the data foundation for that reasoning — a cache that, whenever the leakage steps, looks back over recent energy activity and answers “*what energy event happened when the leakage changed?*”.

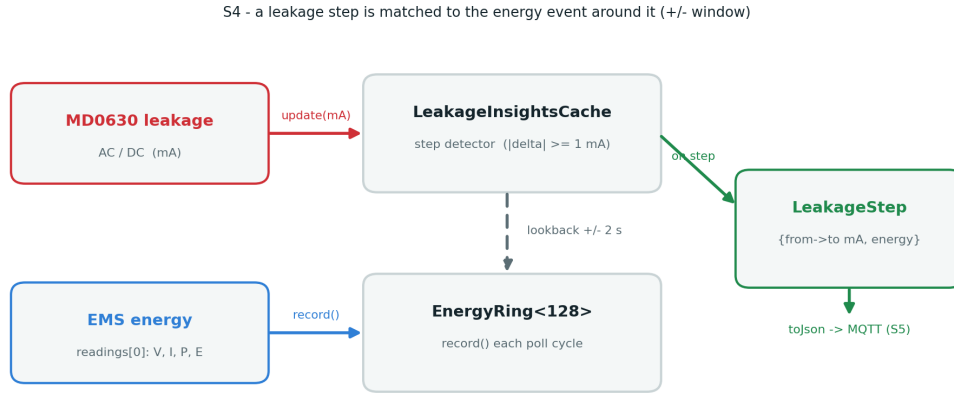


Figure 4: S4 data flow — a leakage step is matched to the energy event around it ($\pm$ window).

5.2 Design

Two independent streams meet in the cache (Figure 4). The **EMS energy** (`readings[0]`) is continuously recorded into a ring buffer of 128 timestamped snapshots, each deriving the active-power **ramp** against the previous sample. The **leakage reading** drives a step detector that fires when $|\Delta| \geq 1$ mA; on a step it queries the ring for the nearest snapshot within a ± 2 s window and attaches it to the event.

Three design choices reflect the domain:

- **Change-based, not level-based.** Acting on *steps* — not the always-present standing residual — avoids false triggers, matching the change-trigger principle later confirmed in the S5 literature.
- **Generic energy ring**, so the same pattern is reusable for other insights (neutral balance, harmonics) — aligning with the supervisor’s repeatable *Insights-Manager* framework.
- A reserved **phase_angle_deg** field: zero today (the MD0630 is absolute-value only), but exactly the quantity a future RCM upgrade fills to obtain **leakage direction** (see S5).

S4 — key result

An energy-change correlation cache (`EnergyRing`, `LeakageInsightsCache`) links each leakage step to the surrounding energy event. It compiles and is exercised end-to-end by the mock ramp; it is the structured input the S5 manager consumes.

6 S5 — Multi-EMS nomination and MQTT isolation

6.1 Objective and literature grounding

When several EMS share a feeder, a single leak is visible — at different magnitudes — to several of them. S5 answers *which* EMS is nearest the fault and how to isolate it. The scheme is deliberately **grounded in the fault-location literature** rather than in intuition; two open-access sources were read in full [4, 5] and a phase-angle criterion extracted [6]:

- **Nominate the “odd one out”, not the loudest.** Faulty-feeder identification compares the *trend correlation* of the transient across feeders and selects the one that **correlates least** with its peers (faulted = $\arg \min_k \mu_k$) [4]. Because EMS sit at different feeder positions, their absolute levels are heterogeneous; trend-correlation is robust to that. The design therefore ranks by **magnitude and by lowest peer-correlation**, gated by a confidence score.
- **Trigger on change, not level** [4] — consistent with S4.
- **Isolate on both sides.** FLISR practice opens switches on **both** sides of the faulted section so only the damaged segment is de-energised, running either fully automatic or operator-approved [5] — mapped here to bracketing the two EMS around the leak, with the auto-vs-alert decision set by the confidence gate.
- **Direction by phase angle.** The residual-voltage/residual-current criterion $|\angle V_{res} - \angle I_{res} - 90^\circ|$ distinguishes upstream from downstream [6] — the principled basis for the reserved `phase_angle_deg` field and the RCM-upgrade path.

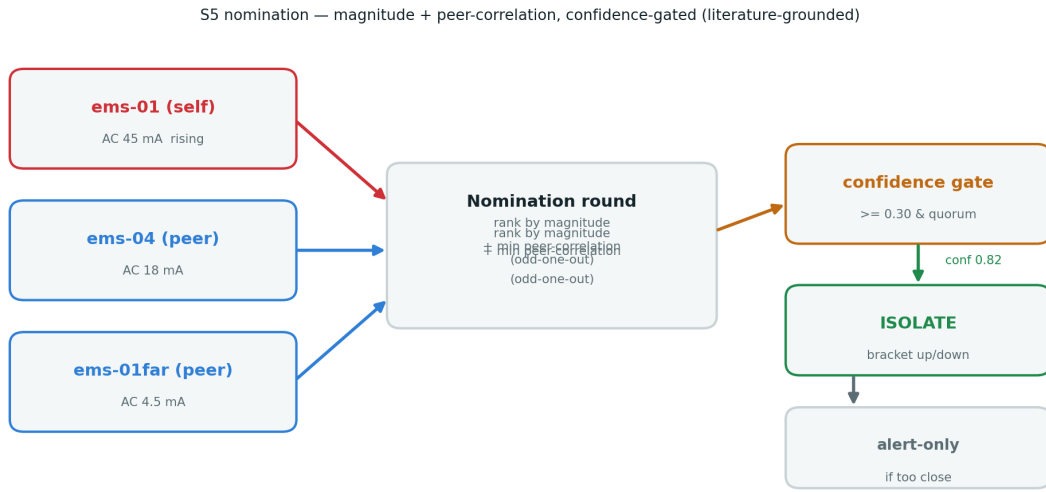


Figure 5: S5 nomination flow — rank by magnitude and by minimum peer-correlation (odd-one-out), confidence-gated to auto-isolate or alert-only.

6.2 Implementation and evidence

The `LeakageInsightsManager` implements a per-EMS state machine (`normal` → `watch` → `isolating` → `fault` → `restored`), a ramp **signature** for correlation, and the three MQTT payloads (telemetry, isolation round, alert). Nomination combines magnitude and minimum peer-correlation and **boosts confidence when the magnitude leader is also the odd-one-out**, de-rating it under reverse power flow.

Run on **one ESP32** with two simulated peers (mid $\approx 0.4\times$, far $\approx 0.1\times$), the nominated node is both the magnitude leader *and* the least-correlated node, giving confidence **0.82** and an auto-isolate decision. The leakage was then streamed over **MQTT to a broker on the PC** over a real WiFi network — and charted live as the mock ramp rose and held at its ceiling (figures below).

Figure: Live leakage telemetry charted in MQTT Explorer (`rcm_ac_mA` / `rcm_dc_mA`).

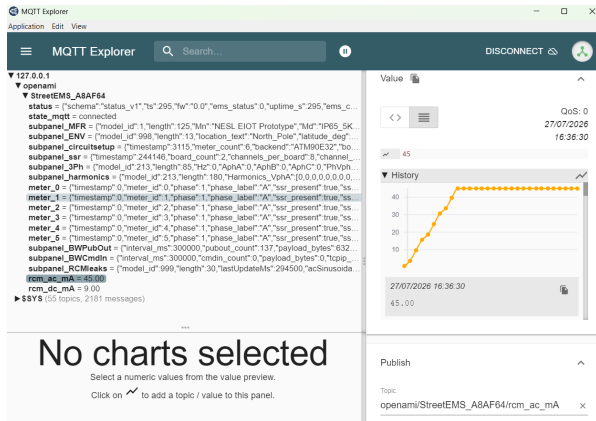


Figure 6: AC leakage rising to 45 mA then holding.

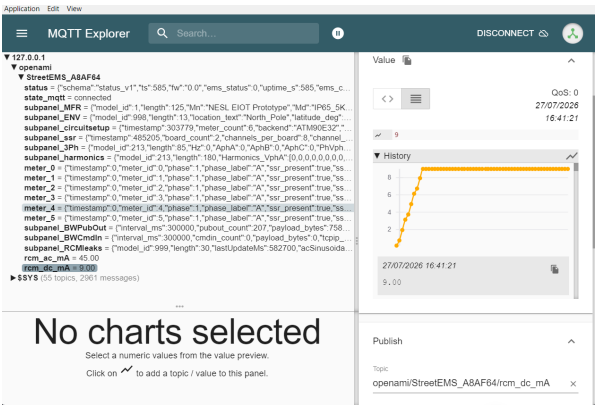


Figure 7: DC leakage rising to 9 mA then holding.

S5 — key result

A literature-grounded nomination (magnitude *and* peer-correlation, confidence-gated) locates a leak across three EMS on one ESP32, and the live leakage streams over MQTT with fault flag and live charts. A WPA3/PMF hotspot blocked the ESP32’s older WiFi stack; a WPA2 hotspot connected immediately — a documented deployment constraint.

7 S6 — Hardware alarm sense and reverse-flow-adaptive protection

S6 closes the loop between the module and the EMS in two directions (Figure 8): the EMS **observes** the module’s own hardware alarm (A), and it **adapts** the AC threshold to the site’s power-flow direction (B).

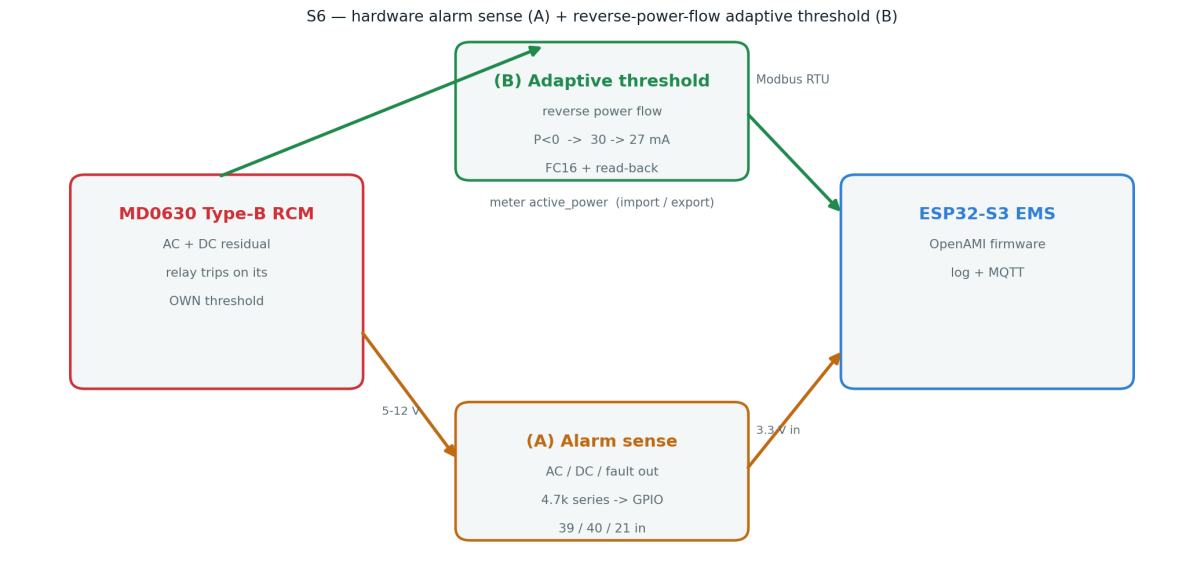


Figure 8: S6 architecture — hardware alarm sense (A) and reverse-power-flow adaptive threshold (B) between the MD0630 and the EMS.

7.1 (B) Reverse-power-flow adaptive threshold

Under **export** (PV/battery back-feed) the acceptable leakage budget changes, so the AC limit is tightened **30** → **27 mA** and restored on return to import. Export is detected from the meter's active power (< -50 W). Crucially, the threshold is re-written **only on a debounced transition** (three poll cycles) — never every cycle — reusing the S3 FC16 + read-back path so the module's store is protected. The live export flag is also fed into the S5 manager so nomination **de-rates confidence under reverse flow**.

7.2 (A) Hardware alarm sense — and its hardware validation

The MD0630 trips its own internal relay when leakage crosses a threshold (the life-safety path, independent of firmware). S6-A lets the EMS **observe** those AC/DC/fault alarm outputs so a trip is logged and published — deliberately **read-only**: it never drives the trip. Each 5 V output reaches a GPIO through a **single 4.7 k Ω series resistor** — a level shift that is safe for *any* output type without a multimeter (an open-collector reads low when active; a push-pull 5 V high is clamped by the GPIO's internal ESD diode with the resistor limiting the injected current to ≈ 0.36 mA).

Flashing this on the bench turned into the most instructive part of the whole assignment: **three defects surfaced that only real hardware reveals**, and each was fixed and re-verified.

Table 6: S6-A defects found and fixed during on-hardware verification.

Defect found on hardware	Diagnosis	Fix
Chosen pins GPIO 33/34 absent on the devkit	not broken out on the ESP32-S3-DevKitC-1 (they were custom-PCB pins)	move to GPIO 39/40/21 (free header pins)
False AC=1 DC=1 at rest	outputs sit low at rest, high on alarm — i.e. active-high, not active-low	<code>active_low=false + INPUT_PULLDOWN</code>
Manual pulse missed by the logger	alarm sampled only at the ~seconds Modbus poll	fast <code>service_leakage_alarms()</code> at ~50 Hz, decoupled

After the fixes, the rest state reads correctly quiet and a stimulus on the AC line is logged live:

```
SETUP: MODBUS: MD0630 alarm sense on GPIO 39/40/21 (AC/DC/fault, active-high)
S6 ALARM edge: AC=0 DC=0 FAULT=0      # at rest, no leakage - quiet
S6 ALARM edge: AC=1 DC=0 FAULT=0      # A0 asserted (3V3 pulse on GPIO39)
S6 ALARM edge: AC=0 DC=0 FAULT=0      # released - DC/FAULT correctly stay 0
```

S6 — key result

Both paths are implemented and gated behind build flags (the firmware builds cleanly with S6 off and with every S6 path on). **S6-A was verified on hardware end-to-end** — module alarm output → 4.7 k Ω → GPIO → firmware logs AC=1 within < 200 ms — after fixing pin availability, alarm polarity and sampling latency. S6-B reuses the proven S3 write path.

8 Consolidated results and validation

Table 7 collects the verification status of each capability. The distinction between *code-proven* (exercised by the mock pipeline) and *hardware-proven* (validated against the physical module) is kept explicit — an honesty the “path of least blockage” method demands.

Table 7: Consolidated verification status across S1–S6.

Capability	Verification	Status
Modbus link + register map (S1)	live device + vendor CT.exe frames	hardware-proven
Live AC/DC read (S2)	12/12 reads, 0 failures on the EMS	hardware-proven
Threshold write + read-back (S3)	AC 30\$→27→30mA on the EMS; FC16 traced hardware-proven Energy – change correlation(S4) compiles; step→\$correlation via mock ramp	code-proven
Nomination + MQTT isolation (S5)	3-EMS round on one ESP32; live MQTT + charts	hardware-proven (mock peers)
Reverse-flow threshold (S6-B)	builds; transition-only FC16 write path	code-proven
Hardware alarm sense (S6-A)	live AC=1/AC=0 on the EMS, \$<\$200 ms	hardware-proven

The firmware builds cleanly in both the default configuration (all leakage test flags off — no regression) and with every S1–S6 capability compiled in, at ~19 % RAM and ~28 % flash on the ESP32-S3.

9 Discussion, limitations and perspectives

What the work demonstrates. A commodity Type-B RCM with an undocumented protocol was turned into a fully-integrated, continuously-monitored EMS subsystem: protocol reverse-engineered and confirmed, values read live, thresholds safely written, changes correlated with energy, leaks nominated and isolated across a feeder, and hardware alarms sensed while protection adapts to power-flow direction. Just as important, the process was **honest about assumptions** and **confirmed them on real hardware** — which is exactly what surfaced the FC06/FC16 issue and the three S6 hardware defects.

Limitations. The peers in the S5 nomination are simulated on a single ESP32; a true feeder trial needs several EMS publishing on shared topics. The energy-correlation thresholds (S4) are set against a bare bench and want tuning against real metered load. And the MD0630 reports the leakage **magnitude only** — without phase, the nomination ranks proximity but cannot yet resolve **direction** (upstream vs downstream).

Perspectives. Three lines follow naturally. **(1)** Replace the simulated peers with real multi-EMS MQTT nomination on the shared `lvfeeder/...` topics. **(2)** Complete the S6 bench captures — the reverse-flow 30\$ \$27 mA switch on the live module, and a genuine `$>30mA leak tripping the alarm`. **(3)** `Adopt an RCM exposing the residual — voltage/current phase angle`, which upgrades the magnitude ranking to true `direction` —

basedisolation * *viathe| $V_{\text{res}} - I_{\text{res}} - 90^\circ$ criterion [6] — the field already reserved in the data model since S4.

References

1. IEC 60755, *General requirements for residual current operated protective devices*, International Electrotechnical Commission.
2. IEC 62955, *Residual direct current detecting device (RDC-DD) to be used for mode 3 charging of electric vehicles — Type-B / DC residual-current detection*.
3. Modbus Organization, *MODBUS Application Protocol Specification*, V1.1b3.
4. *Faulty Feeder Identification in a distribution network by data analysis and similarity (trend-correlation) comparison of zero-sequence transients*, PMC7795609 (open access, read in full).
5. *FLISR — Fault Location, Isolation and Service Restoration: workflow and practice* (read).
6. *An additional criterion for faulty-feeder selection during ground faults using the $V_{\text{res}}/I_{\text{res}}$ phase angle ($\approx 90^\circ$)*.
7. IVY Metering, *MD0630 Type-B AC+DC residual-current monitor* — datasheet and `CT.exe` configuration tool (vendor).

\ {Repository: `docs/leakage_MD0630TA1A/` on branch `modbus-testing`. This consolidated report is built from the per-sprint reports (`S1–S6`) and the on-hardware bench evidence (`S6_bench_evidence.md`); all source, drivers and diagrams are version-controlled.}